

NeuFlow v3

Queryable optical flow with calibrated uncertainty

Shriman Raghav Srinivasan

MS Robotics, Northeastern University · Field Robotics Lab · August 2026

Summary

NeuFlow v2 always computes flow for every pixel at one fixed resolution. v3 replaces only its final upsampling stage with an implicit decoder, so the model answers flow queries at any continuous coordinate. Cost scales with the number of points asked for, $O(N)$, not with image area, $O(H \times W)$.

v3 matches v2's accuracy with 13% fewer parameters. It adds three things v2 cannot do at all: flow at sub-pixel coordinates, repeat queries on a cached frame at $7.7\times$ lower cost, and a calibrated confidence value per query.

1. Why queryable flow

Flow networks produce a dense map. Many consumers do not want one.

- Registration needs a few hundred correspondences at points it picks itself.
- Sparse tracking needs flow only at feature points.
- Survey mosaicking needs matches in overlap regions, often between pixels.

Computing 479,232 values to use 800 of them is the difference between real time and not on constrained hardware. A fixed-resolution design also cannot answer any query above its input sampling rate, which is what high-resolution registration needs.

The question this project asks: can the last stage be made queryable without losing accuracy or speed?

2. Method

2.1 What is inherited

NeuFlow v2 (Zhang, Gupta, Jiang & Singh, arXiv:2408.10161) runs a shallow CNN backbone at $1/8$ and $1/16$ resolution, cross-attention and global matching at $1/16$ for an initial flow, recurrent refinement (one iteration at $1/16$, eight at $1/8$), then a learned convex upsampler to full resolution.

v3 keeps all of that up to the $1/8$ coarse flow, with every learned weight frozen. Only the upsampler is replaced. In the runs reported here the batch-normalisation running statistics of the frozen stack adapted to the training data instead of holding v2's values. Section 6 covers what that costs.

2.2 The decoder

Two phases.

Phase 1, once per frame pair, 16.6 ms. The frozen pipeline produces the coarse flow and feature maps. These are cached.

Phase 2, once per query batch, 2.55 ms for up to 2,048 points. For a coordinate (x, y) the decoder samples 3×3 windows from four sources ($1/8$ context, $1/8$ features, $1/16$ features, flow-

warped frame-1 features), fuses them in a gated MLP, and predicts weights over ten candidates: the nine coarse-flow values around the query plus a bilinear sample.

Output is a convex combination of those candidates. That bounds the prediction inside the locally supported motion range, so the decoder cannot invent displacement its inputs do not support. The head is initialised so the softmax sits on the bilinear candidate, which makes the untrained decoder exactly equivalent to bilinear upsampling (measured: 0.011 px maximum deviation). Training can only move away from that by learning.

2.3 Uncertainty head

An optional eleventh output predicts a per-query error scale b under a Laplace likelihood. It costs no measurable inference time and attaches a confidence value to every correspondence returned.

2.4 Interface

```
state = model.infer_coarse_state(img0, img1)          # once per pair
flow  = model.decode_queries(state, query_coords=q)   # [B,N,2] -> [B,N,2]
flow  = model.decode_queries(state, target_h=H, target_w=W)
flow, b = model.decode_queries(state, query_coords=q, return_uncertainty=True)
```

Coordinates are continuous. (312.7, 188.2) is as valid as (312, 188). Sparse queries reproduce the dense field exactly at matching coordinates, to 0.00057 px.

3. Setup

Evaluation. VKITTI2 Scene18 and Scene20: 1,174 pairs, 460,573,660 valid pixels, per-pixel metrics. Both scenes are excluded from every training set. A guard in the loader enforces it and I verify no overlap at pair and frame level before each run.

Controls. Every training run is generated from one template, so any two differ in exactly one variable. All use seed 1234, batch 16, learning rate 2×10^{-4} on a OneCycle schedule, $\gamma = 0.8$, 4,096 supervision queries per image, refinement of $1 \times s16$ and $8 \times s8$ matching evaluation, and 100,000 steps.

Pre-flight. A nine-check suite runs on CPU and GPU before any job: batch-normalisation frozen, only decoder parameters trainable, zero-initialised decoder equals bilinear, sparse equals dense, uncertainty present in both decode paths, stride-2 fidelity, seed reproducibility.

Hardware. V100, fp16, 384×1248 input unless stated.

4. Results

4.1 Accuracy

Model	Training data	EPE (px)	1px (%)	3px (%)	Params
NeuFlow v2	FlyingThings	2.324	77.63	89.80	9.03 M
NeuFlow v3	FlyingChairs	2.286	71.30	87.57	7.83 M
NeuFlow v3	+ VKITTI2	2.138	76.38	89.46	7.83 M
NeuFlow v3	+ MPI-Sintel	2.147	76.81	89.56	7.83 M
NeuFlow v3	+ uncertainty head	2.104	76.88	89.61	7.83 M

The FlyingChairs row is the fair comparison. That model saw no driving imagery, which is also true of v2. It scores 2.286 px against v2’s 2.324 px on unseen driving scenes, so the two are equivalent, with v3 using 13% fewer parameters.

The mixed-data rows reach 2.104 px, but they train on VKITTI2 scenes from the same simulator as the test set and v2 never saw that data. Those numbers show what the architecture reaches given representative data. They are not a fair comparison and I do not present them as beating v2.

The three mixed configurations span 2.10 to 2.16 px and cannot be separated from each other. Evaluating two checkpoints of each run moves the result by up to 0.038 px, which is the same size as the gaps between them. Nothing below roughly 0.05 px is resolvable here. What does hold at both checkpoints: adding in-domain data is worth about 0.15 px and five points of 1-pixel accuracy.

4.2 Compute

Mode	Latency	Note
NeuFlow v2, full frame	19.6 ms	its only mode
v3 sparse, first query on a new pair	19.16 ms	level with v2
v3 sparse, repeat query on a cached pair	2.55 ms	7.7× cheaper
v3 dense, stride 2	22.0 ms	not the intended mode

The sparse path is a 16.61 ms coarse pass inherited from v2 plus a 2.55 ms decode. Global matching needs whole-image context, so the coarse pass cannot be skipped. It is 87% of a first query, which is why a first query costs what a v2 frame costs.

State reuse is the property that matters. v2 keeps nothing between calls, so asking a second question about the same frame means recomputing everything. v3 answers from cached state in 2.55 ms. For anything that queries a frame more than once (iterative registration, RANSAC refinement, interactive selection, multi-hypothesis tracking) that is a structural difference, not a margin.

Decode latency is 2.553 ms at $N = 800$ and 2.554 ms at $N = 2,048$. The decode is bound by kernel launch overhead, not compute, so 2,048 points cost the same as 800.

4.3 Calibrated uncertainty

Predicted scale b	Actual mean error
0.01 to 0.11	0.313 px
0.11 to 0.20	0.504 px
0.20 to 0.41	0.807 px
0.41 to 1.22	1.652 px
above 1.22	6.723 px

Measured over 2,348,000 queries. Actual error rises monotonically across all five bins, a $21\times$ span, Pearson $r = 0.345$. The correlation is not strong but the signal is clearly informative and directly usable: weight correspondences in RANSAC, drop unreliable ones, or send more queries where the model is unsure. v2 outputs flow only and has nothing comparable.

4.4 Interactive tool

A PyQt application loads a video, steps frame by frame, and computes flow for a region the user drags out. Two modes: exact decoding inside the box against a full-frame coarse pass, or a cropped mode running the whole pipeline on the selection so cost scales with the area requested. Both print their latency breakdown live.

5. What v3 is for

The contribution is an operating point, not a leaderboard position. v3 gives v2’s accuracy from a smaller model and adds three properties the fixed-resolution design cannot express:

1. Flow at any continuous coordinate, with sparse output identical to dense.
2. Repeat access at 2.55 ms per query batch against a cached frame, versus a full 19.6 ms recomputation.
3. A calibrated error estimate on every correspondence.

If an application consumes one full dense flow field per frame and nothing else, v2 is the better choice. It produces that product 12% faster and with higher sub-pixel precision. v3 is the right choice when the consumer picks its own query points, revisits a frame, needs positions between pixels, or wants a confidence value.

6. Limitations

Sub-pixel precision. v3 trails v2 by 0.8 to 6.3 points of 1-pixel accuracy while matching or beating it on mean error. The decoder makes fewer large errors but is less exact on most pixels. I know the cause: the decoder’s finest input is at 1/8 resolution, so the evidence it sees barely changes inside an 8×8 cell, while v2’s upsampler reads the full-resolution frame directly. Adding a Fourier positional encoding of the sub-cell offset changed nothing, which rules out missing positional signal and points at missing high-resolution features.

Adapted normalisation statistics. The frozen stack’s learned weights are unchanged, but its batch-normalisation running statistics took roughly 24,800 updates during training instead of holding v2’s values. Two consequences. The comparison in Section 4.1 is v3 with statistics adapted to its own training set against v2 with statistics from FlyingThings, not a decoder-only comparison, and for the mixed runs that adaptation points at the evaluation domain. Out-of-domain robustness also suffers: on 1920×1080 Spring imagery, far from the 320×496 training crops, the adapted statistics degrade the coarse flow badly while v2’s untouched statistics do not. The training loop is fixed. Runs with strictly frozen statistics are needed to separate the decoder’s contribution from this effect.

No embedded measurement. Every latency figure is V100 or RTX 4060. The edge claim is a design target, not a result.

One evaluation domain. VKITTI2 is synthetic driving footage. It says little about field or survey imagery.

Statistical resolution. One seed per configuration, with checkpoint-to-checkpoint variation up to 0.038 px. Differences below about 0.05 px are not resolved.

7. Next

1. **Full-resolution stem.** Give the decoder a cheap full-resolution feature map, estimated 1 to 2 ms. This attacks the precision limitation directly and makes sub-pixel querying substantive rather than interpolative.
2. **Re-run with frozen statistics,** removing the confound in Section 6 and restoring out-of-domain robustness.
3. **Decode-path optimisation.** The decode is launch-bound. CUDA graph capture, fusing the three samplers that share coordinates, and `torch.compile` (measured at 9% on the coarse pass) together project a first query below v2’s latency.
4. **Spring high-resolution evaluation,** testing output above the input sampling rate. Blocked

on item 2.

5. **Jetson measurement**, which converts the edge claim into a result or refutes it.
6. **Registration on field imagery**, testing the intended application.

Code, training configurations and the full development record: github.com/shrirag10/Neuflowv3. Every number here is reproducible from `scripts/eval_all_runs.py`, `scripts/benchmark_sparse.py` and `scripts/eval_calibration.py`.